

AI-TECH

Przykład 1 DO LABORATORIUM 5 z UCZENIA MASZYNOWEGO

Sztuczne sieci neuronowe

Tomasz Neumann

Klasyfikacja wieloklasowa, wieloetykietowa i
wielowyjściowa w Scikit-Learn

5. Przykład 1

Klasyfikatory wieloklasowe (wielomianowe) służą do rozróżniania większej liczby klas. Niektóre algorytmy (takie jak klasyfikatory SGD, klasyfikatory losowego lasu lub naiwne klasyfikatory bayesowskie) są w stanie natywnie zajmować się wieloma klasami naraz. W przypadku wektorów nośnych czy klasyfikatorów regresji logistycznej mają charakter typowo binarny. Istnieje jednak wiele strategii, dzięki którym możemy przeprowadzać klasyfikację wieloklasową przy użyciu wielu klasyfikatorów binarnych.

Jednym ze sposobów stworzenia systemu klasyfikującego 10 klas obrazów cyfr (od 0 do 9) polega na wyuczeniu 10 klasyfikatorów binarnych, po jednym na każdą cyfrę. Następnie na etapie klasyfikacji obrazu uzyskujemy wynik z każdego klasyfikatora dla danej cyfry i wybieramy klasę, która uzyskała najwyższy wynik. Jest to tzw. strategia jeden przeciw reszcie (*one-versus-rest* — *OvR* lub *one-versus-all* — *OvA*). Jeszcze innym rozwiązaniem jest wyuczenie klasyfikatora binarnego dla każdej pary cyfr: jeden rozpoznaje zera i jedynki, drugi — zera i dwójki, jeszcze inny — jedynki i dwójki itd. Jest to strategia jeden przeciw jednemu (*one-versus-one* — *OvO*). Jeśli masz do czynienia z N klasami, musisz wytrenować $N \times (N-1)/2$ klasyfikatorów. W przypadku zestawu danych MNIST oznacza to wyuczenie 45 klasyfikatorów binarnych.

Zaletą strategii *OvO* jest fakt, że każdy klasyfikator musi zostać wytrenowany jedynie wobec części zbioru uczącego składającego się z obydwu porównywanych klas. Pewne algorytmy nie skalują się zbyt dobrze do rozmiarów zbioru uczącego np. maszyna wektorów nośnych. W takim przypadku preferowane jest stosowanie strategii *OvO*, ponieważ uczenie wielu klasyfikatorów wobec niewielkich zbiorów danych przebiega szybciej niż trenowanie kilku klasyfikatorów wobec dużych zbiorów przykładów. Jednakże w przypadku większości klasyfikatorów binarnych zalecane jest korzystanie ze strategii *OvR*.

Moduł Scikit-Learn pozwala zastosować strategię *OvO* lub *OvR*, poprzez wykorzystanie klas *OneVsOneClassifier* lub *OneVsRestClassifier*. Wystarczy stworzyć ich wystąpienie i przekazać konstruktorowi klasyfikator (nie musi być on nawet binarny). Poniższy kod przykładu stworzy klasyfikator wieloklasowy (na podstawie klasyfikatora *SVC*) przy użyciu strategii *OvR*.

```
In [1]: #import bibliotek i danych
import keras
import tensorflow as tf
import numpy as np

(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data(
y_train = y_train.astype(np.uint8)
y_test = y_test.astype(np.uint8)
X_train = X_train.reshape(X_train.shape[0],X_train.shape[1]*X_train.shape
X_test = X_test.reshape(X_test.shape[0],X_test.shape[1]*X_test.shape[2])
#zainicjowanie modelu OvR
#### UWAGA: trening trwa bardzo długo dlatego zaleca się jego wykonanie p
#### W celu testów odkomentuj poniższe linijki
from sklearn.multiclass import OneVsRestClassifier
from sklearn import svm
ovr_clf = OneVsRestClassifier(svm.SVC())
ovr_clf.fit(X_train, y_train)
print(ovr_clf.predict(X_train))
```

```
[5 0 4 ... 5 6 8]
```

Podobnie można postąpić z klasyfikatorem SGDClassifier

```
In [2]: from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier()
sgd_clf.fit(X_train, y_train)
print(sgd_clf.predict(X_train))
```

```
[5 0 3 ... 5 6 8]
```

Klasyfikatory SGD mogą bezpośrednio klasyfikować przykłady w sposób wieloklasowy, stąd też nie ma potrzeby wdrażania powyżej opisanych strategii. W takim zastosowaniu metoda `decision_function()` zwraca aktualnie po jednej wartości na jedną klasę.

```
In [3]: i=1244
test0 = X_train[i].reshape(1,784)
print(test0.shape)
print(sgd_clf.predict(test0))
print(sgd_clf.decision_function(X_train)[i][:])
print('Przewidywana etykieta: ', sgd_clf.predict(X_train[i].reshape(1,784)
print('Prawdziwa etykieta: ', y_train[i])
```

```
(1, 784)
```

```
[9]
```

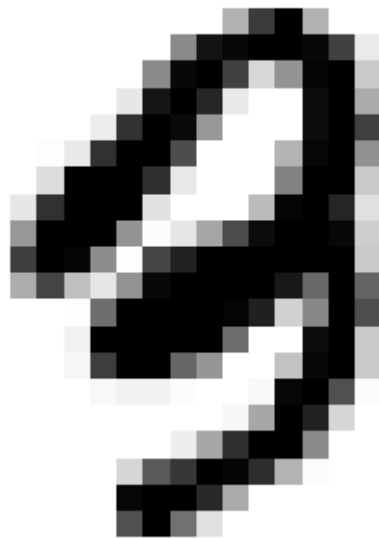
```
[-25680.12792537 -34809.53067135 -9061.25559391 -14594.8093721
-4784.98265286 -13833.47607736 -17879.59383141 -10787.88406468
-181.27072667 1090.17752807]
```

```
Przewidywana etykieta: 9
```

```
Prawdziwa etykieta: 3
```

Jak jednak przyjrzymy się dokładniej, przy wybraniu tego obrazu do przewidywań, klasyfikator pomylił się w ocenie, wykazując największy stopień pewności wyniku dla cyfry 9, a natępnie drugą wartość dla cyfry 8, która jest także wartością nieprawidłową. Jak widać wartość przewidywań dla cyfry 3 jest dużą liczbą ujemną stąd klasyfikator nie będzie brał tej opcji do końcowych przewidywań.

```
In [4]: import matplotlib as mpl
import matplotlib.pyplot as plt
some_digit_image = X_train[i].reshape(28,28)
plt.imshow(some_digit_image, cmap = "binary")
plt.axis("off")
plt.show()
print('Prawdziwa etykieta: ', y_train[i])
```



Prawdziwa etykieta: 3

Był to jednak specjalnie wybrany przykład niepoprawnej klasyfikacji. W celu oceny skuteczności tego klasyfikatora można posłużyć się sprawdzianem krzyżowym z 5-cio krotną krosvalidacją.

```
In [5]: from sklearn.model_selection import cross_val_score
cross_val_score(sgd_clf, X_train, y_train, cv=5, scoring="accuracy")
```

KeyboardInterrupt

Traceback (most recent call last)

Cell In[5], line 2

```
1 from sklearn.model_selection import cross_val_score
----> 2 cross_val_score(sgd_clf, X_train, y_train, cv=5, scoring=
    )
```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils_param_validation.py:218, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)

```
212 try:
213     with config_context(
214         skip_parameter_validation=(
215             prefer_skip_nested_validation or global_skip_validation
n
216         )
217     ):
--> 218         return func(*args, **kwargs)
219 except InvalidParameterError as e:
220     # When the function is just a wrapper around an estimator, we
allow
221     # the function to delegate validation to the estimator, but we
replace
222     # the name of the estimator by the name of the function in the
error
223     # message to avoid confusion.
224     msg = re.sub(
225         r"parameter of \w+ must be",
226         f"parameter of {func.__qualname__} must be",
227         str(e),
228     )
```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\model_selection_validation.py:677, in cross_val_score(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, error_score)

```
674 # To ensure multimetric format is not supported
675 scorer = check_scoring(estimator, scoring=scoring)
--> 677 cv_results = cross_validate(
678     estimator=estimator,
679     X=X,
680     y=y,
681     groups=groups,
682     scoring={          : scorer},
683     cv=cv,
684     n_jobs=n_jobs,
685     verbose=verbose,
686     params=params,
687     pre_dispatch=pre_dispatch,
688     error_score=error_score,
689 )
690 return cv_results["test_score"]
```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils_param_validation.py:218, in validate_params.<locals>.decorator.<locals>.wrapper(*args, **kwargs)

```
212 try:
213     with config_context(
214         skip_parameter_validation=(
```

```

215         prefer_skip_nested_validation or global_skip_validation
n
216     )
217 ):
--> 218     return func(*args, **kwargs)
219 except InvalidParameterError as e:
220     # When the function is just a wrapper around an estimator, we
allow
221     # the function to delegate validation to the estimator, but we
replace
222     # the name of the estimator by the name of the function in the
error
223     # message to avoid confusion.
224     msg = re.sub(
225         r"parameter of \w+ must be",
226         f"parameter of {func.__qualname__} must be",
227         str(e),
228     )

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\model_selection_validation.py:399, in cross_validate(estimator, X, y, groups, scoring, cv, n_jobs, verbose, params, pre_dispatch, return_train_score, return_estimator, return_indices, error_score)

```

396 # We clone the estimator to make sure that all the folds are
397 # independent, and that it is pickle-able.
398 parallel = Parallel(n_jobs=n_jobs, verbose=verbose, pre_dispatch=p
re_dispatch)
--> 399 results = parallel(
400     delayed(_fit_and_score)(
401         clone(estimator),
402         X,
403         y,
404         scorer=scorers,
405         train=train,
406         test=test,
407         verbose=verbose,
408         parameters=None,
409         fit_params=routed_params.estimator.fit,
410         score_params=routed_params.scorer.score,
411         return_train_score=return_train_score,
412         return_times=True,
413         return_estimator=return_estimator,
414         error_score=error_score,
415     )
416     for train, test in indices
417 )
419 _warn_or_raise_about_fit_failures(results, error_score)
421 # For callable scoring, the return type is only know after callin
g. If the
422 # return type is a dictionary, the error scores can now be inserte
d with
423 # the correct key.

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils\parallel.py:82, in Parallel.__call__(self, iterable)

```

73 warning_filters = warnings.filters
74 iterable_with_config_and_warning_filters = (
75     (
76         _with_config_and_warning_filters(delayed_func, config, war
ning_filters),

```

```

(...)      80      for delayed_func, args, kwargs in iterable
      81 )
--> 82 return super().__call__(iterable_with_config_and_warning_filters)

File c:\Users\igor\anaconda3\Lib\site-packages\joblib\parallel.py:1986, in
Parallel.__call__(self, iterable)
    1984     output = self._get_sequential_output(iterable)
    1985     next(output)
-> 1986     return output if self.return_generator else list(output)
    1988 # Let's create an ID that uniquely identifies the current call. If
the
    1989 # call is interrupted early and that the same instance is immediat
ely
    1990 # reused, this id will be used to prevent workers that were
    1991 # concurrently finalizing a task from the previous call to run the
    1992 # callback.
    1993 with self._lock:

File c:\Users\igor\anaconda3\Lib\site-packages\joblib\parallel.py:1914, in
Parallel._get_sequential_output(self, iterable)
    1912 self.n_dispatched_batches += 1
    1913 self.n_dispatched_tasks += 1
-> 1914 res = func(*args, **kwargs)
    1915 self.n_completed_tasks += 1
    1916 self.print_progress()

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils\parallel.py:1
47, in _FuncWrapper.__call__(self, *args, **kwargs)
    145 with config_context(**config), warnings.catch_warnings():
    146     warnings.filters = warning_filters
--> 147     return self.function(*args, **kwargs)

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\model_selection\_va
lidation.py:859, in _fit_and_score(estimator, X, y, scorer, train, test, v
erbose, parameters, fit_params, score_params, return_train_score, return_p
arameters, return_n_test_samples, return_times, return_estimator, split_pr
ogress, candidate_progress, error_score)
    857     estimator.fit(X_train, **fit_params)
    858     else:
--> 859     estimator.fit(X_train, y_train, **fit_params)
    861 except Exception:
    862     # Note fit time as time until error
    863     fit_time = time.time() - start_time

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\base.py:1365, in _f
it_context.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
    1358     estimator._validate_params()
    1360 with config_context(
    1361     skip_parameter_validation=(
    1362         prefer_skip_nested_validation or global_skip_validation
    1363     )
    1364 ):
-> 1365     return fit_method(estimator, *args, **kwargs)

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\linear_model\_stoch
astic_gradient.py:920, in BaseSGDClassifier.fit(self, X, y, coef_init, int
ercept_init, sample_weight)
    891 """Fit linear model with Stochastic Gradient Descent.
    892
    893 Parameters

```

```

(...) 916     Returns an instance of self.
917 """
918 self._more_validate_params()
--> 920 return self._fit(
921     X,
922     y,
923     alpha=self.alpha,
924     C=1.0,
925     loss=self.loss,
926     learning_rate=self.learning_rate,
927     coef_init=coef_init,
928     intercept_init=intercept_init,
929     sample_weight=sample_weight,
930 )

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\linear_model_stochastic_gradient.py:707, in BaseSGDClassifier._fit(self, X, y, alpha, C, loss, learning_rate, coef_init, intercept_init, sample_weight)

```

704 # Clear iteration count for multiple call to fit.
705 self.t_ = 1.0
--> 707 self._partial_fit(
708     X,
709     y,
710     alpha,
711     C,
712     loss,
713     learning_rate,
714     self.max_iter,
715     classes,
716     sample_weight,
717     coef_init,
718     intercept_init,
719 )
721 if (
722     self.tol is not None
723     and self.tol > -np.inf
724     and self.n_iter_ == self.max_iter
725 ):
726     warnings.warn(
727         (
728             "Maximum number of iteration reached before "
729             "
730             "
731             "
732             ConvergenceWarning,
733     )

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\linear_model_stochastic_gradient.py:641, in BaseSGDClassifier._partial_fit(self, X, y, alpha, C, loss, learning_rate, max_iter, classes, sample_weight, coef_init, intercept_init)

```

639 # delegate to concrete training procedure
640 if n_classes > 2:
--> 641     self._fit_multiclass(
642         X,
643         y,
644         alpha=alpha,
645         C=C,
646         learning_rate=learning_rate,
647         sample_weight=sample_weight,
648         max_iter=max_iter,
649     )
650 elif n_classes == 2:

```



```

651     self._fit_binary(
652         X,
653         y,
654     (...) 658         max_iter=max_iter,
659     )

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\linear_model_stochastic_gradient.py:786, in BaseSGDClassifier._fit_multiclass(self, X, y, alpha, C, learning_rate, sample_weight, max_iter)

```

784 random_state = check_random_state(self.random_state)
785 seeds = random_state.randint(MAX_INT, size=len(self.classes_))
--> 786 result = Parallel(
787     n_jobs=self.n_jobs, verbose=self.verbose, require=
788 )((
789     delayed(fit_binary)(
790         self,
791         i,
792         X,
793         y,
794         alpha,
795         C,
796         learning_rate,
797         max_iter,
798         self._expanded_class_weight[i],
799         1.0,
800         sample_weight,
801         validation_mask=validation_mask,
802         random_state=seed,
803     )
804     for i, seed in enumerate(seeds)
805 )
807 # take the maximum of n_iter_ over every binary fit
808 n_iter_ = 0.0

```

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils\parallel.py:82, in Parallel.__call__(self, iterable)

```

73 warning_filters = warnings.filters
74 iterable_with_config_and_warning_filters = (
75     (
76         _with_config_and_warning_filters(delayed_func, config, warn
77 ing_filters),
78 (...) 80         for delayed_func, args, kwargs in iterable
79     )
81 )
--> 82 return super().__call__(iterable_with_config_and_warning_filters)

```

File c:\Users\igor\anaconda3\Lib\site-packages\joblib\parallel.py:1986, in Parallel.__call__(self, iterable)

```

1984     output = self._get_sequential_output(iterable)
1985     next(output)
-> 1986     return output if self.return_generator else list(output)
1988 # Let's create an ID that uniquely identifies the current call. If
the
1989 # call is interrupted early and that the same instance is immediat
ely
1990 # reused, this id will be used to prevent workers that were
1991 # concurrently finalizing a task from the previous call to run the
1992 # callback.
1993 with self._lock:

```

File c:\Users\igor\anaconda3\Lib\site-packages\joblib\parallel.py:1914, in

```

Parallel._get_sequential_output(self, iterable)
    1912 self.n_dispatched_batches += 1
    1913 self.n_dispatched_tasks += 1
-> 1914 res = func(*args, **kwargs)
    1915 self.n_completed_tasks += 1
    1916 self.print_progress()

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\utils\parallel.py:1
47, in _FuncWrapper.__call__(self, *args, **kwargs)
    145 with config_context(**config), warnings.catch_warnings():
    146     warnings.filters = warning_filters
--> 147     return self.function(*args, **kwargs)

File c:\Users\igor\anaconda3\Lib\site-packages\sklearn\linear_model\_stoch
astic_gradient.py:465, in fit_binary(est, i, X, y, alpha, C, learning_rat
e, max_iter, pos_weight, neg_weight, sample_weight, validation_mask, rando
m_state)
    462 tol = est.tol if est.tol is not None else -np.inf
    464 _plain_sgd = _get_plain_sgd_function(input_dtype=coef.dtype)
--> 465 coef, intercept, average_coef, average_intercept, n_iter_ = _plain
_sgd(
    466     coef,
    467     intercept,
    468     average_coef,
    469     average_intercept,
    470     est._loss_function_,
    471     penalty_type,
    472     alpha,
    473     C,
    474     est._get_ll_ratio(),
    475     dataset,
    476     validation_mask,
    477     est.early_stopping,
    478     validation_score_cb,
    479     int(est.n_iter_no_change),
    480     max_iter,
    481     tol,
    482     int(est.fit_intercept),
    483     int(est.verbose),
    484     int(est.shuffle),
    485     seed,
    486     pos_weight,
    487     neg_weight,
    488     learning_rate_type,
    489     est.eta0,
    490     est.power_t,
    491     0,
    492     est.t_,
    493     intercept_decay,
    494     est.average,
    495 )
    497 if est.average:
    498     if len(est.classes_) == 2:

```

KeyboardInterrupt:

Jak widać, wobec wszystkich podzbiorów testowych uzyskuję się wynik 87%, co porównując to wyniku losowego klasyfikatora (10%) wypada dobrze. Aby uzyskać

lepszy wynik można wykorzystać przeskalowanie danych za pomocą klasy *StandardScaler*

```
In [ ]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train.astype(np.float64))
cross_val_score(sgd_clf, X_train_scaled, y_train, cv=5, scoring="accuracy")
```

```
c:\users\tomas\ella\lib\site-packages\sklearn\linear_model\_stochastic_gradient.py:577: ConvergenceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve the fit.
ConvergenceWarning)
```

```
Out[ ]: array([0.90316667, 0.89566667, 0.90316667, 0.89391667, 0.89758333])
```

uzyskując o około 3% lepszy wynik niż wcześniej bez skalowania.

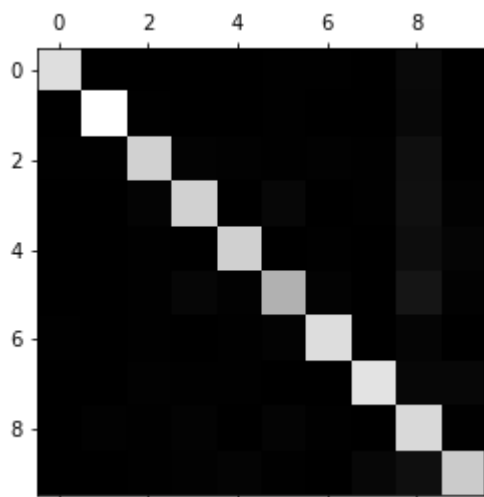
Dalszą analizę wyników można dokonywać za pomocą macierzy pomyłek. Należy wyliczyć prognozy za pomocą funkcji *cross_val_predict()*, a następnie wywołać funkcję *confusion_matrix()* (rzędy reprezentują rzeczywiste klasy, kolumny zaś - przewidywane klasy)

```
In [43]: from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=5)
conf_mx = confusion_matrix(y_train, y_train_pred)
print(conf_mx)
```

```
[5568  0  16  7  9  41  35  6 240  1]
[  0 6409 42 25  3 41  4  7 203  8]
[ 26 26 5242 86 66 26 62 38 376 10]
[ 24 18 112 5234 0 195 24 40 418 66]
[ 11 14 39 13 5215 10 32 20 339 149]
[ 25 18 27 161 53 4454 78 15 530 60]
[ 28 17 45 3 41 88 5545 8 143 0]
[ 18 12 53 26 49 13 4 5697 196 197]
[ 15 60 43 90 3 121 28 8 5448 35]
[ 21 19 32 61 124 38 1 179 388 5086]]
```

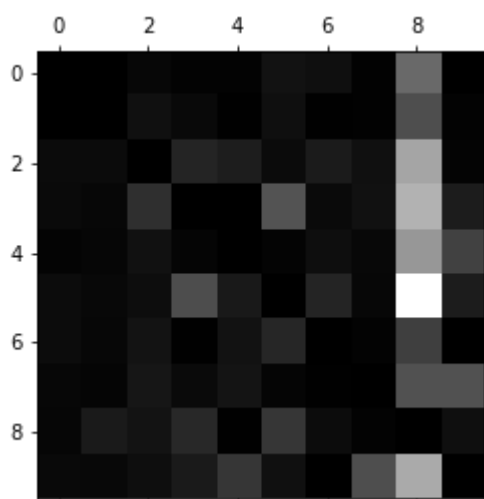
jak i stworzyć reprezentację graficzną tych liczb.

```
In [44]: plt.matshow(conf_mx, cmap=plt.cm.gray)
plt.show()
```



Wartości w macierzy pomyłek prezentują się całkiem nieźle, gdyż większość obrazów znajduje się na głównej przekątnej, co oznacza, że zostały prawidłowo sklasyfikowane. Piątki wyglądają nieco ciemniej od pozostałych cyfr, co może oznaczać, że w zbiorze danych znajduje się mniej obrazów piątek lub że klasyfikator nie sprawuje się tak dobrze wobec piątek, jak w przypadku pozostałych cyfr. W rzeczywistości można zweryfikować obydwa stwierdzenia, jeżeli wyrysuje się diagram błędów.

```
In [45]: #podzielenie każdej wartości z macierzy pomyłek przez liczbę obrazów
row_sums = conf_mx.sum(axis=1, keepdims=True)
norm_conf_mx = conf_mx / row_sums
#Wypełnienie głównej przekątnej zerami, aby pozostały same błędy
np.fill_diagonal(norm_conf_mx, 0)
plt.matshow(norm_conf_mx, cmap=plt.cm.gray)
plt.show()
```



Kolumna reprezentująca klasę 8 jest dość jasna, co oznacza, że wiele obrazów zostało nieprawidłowo sklasyfikowanych jako ta właśnie cyfra. Jednak rząd klasy 8 nie wygląda tak źle, co oznacza, że zasadniczo większość ósemek jest prawidłowo sklasyfikowanych. Jak widać, macierz pomyłek nie jest zbyt symetryczna. Można również dostrzec, że trójki i piątki są często mylone (w obydwie strony). Analiza macierzy pomyłek często dostarcza wskazówek pozwalających usprawnić klasyfikator. Można byłoby popracować nad zmniejszeniem liczby fałszywych

ósemek tworząc nowe cechy bądź też dokonać wstępnego przetwarzania obrazów (korzystając z modułów Scikit-Image, Pillow lub OpenCV), aby uwypuklić pewne wzorce.

Klasyfikacja wieloetykietowa (multilabel classification)

Powyższy kod analizował pojedyncze przykłady i przydzielał je wyłącznie do jednej klasy. W niektórych sytuacjach może zachodzić potrzeba, aby klasyfikator wyznaczał wiele klas dla jednego wystąpienia.

W tym celu zaimplementowane zostaną dwie docelowe etykiety dla każdego obrazu cyfry. Niech pierwsza z nich określa, czy jest to duża liczba, zaś druga niech opisuje, że liczba jest nieparzysta. Następnie zostaje utworzone wystąpienie klasyfikatora *KNeighborsClassifier*, po czym jest on uczony za pomocą tablicy zawierającej wiele docelowych elementów

```
In [46]: from sklearn.neighbors import KNeighborsClassifier
y_train_large = (y_train >= 7)
y_train_odd = (y_train % 2 == 1)
y_multilabel = np.c_[y_train_large, y_train_odd]
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
```

```
Out[46]: KNeighborsClassifier()
```

oraz dokonywane są prognozy.

```
In [47]: print(knn_clf.predict(X_train[0:7]))
print('Prawdziwa etykieta: ', y_train[0:7])
```

```
[[False True]
 [False False]
 [False False]
 [False True]
 [ True True]
 [False False]
 [False True]]
```

```
Prawdziwa etykieta: [5 0 4 1 9 2 1]
```

Jak widać prognoza do obrazu pierwszego, reprezentującego 5-tkę zgadza się - nie jest to wielka cyfra (większa od 6) oraz jest to cyfra nieparzysta. Aby jednak lepiej ocenić wydajność klasyfikatora można obliczyć F-Score dla poszczególnych etykiet (zakładając, że każda ma taką samą wagę), a następnie wyliczeniu średniej.

```
In [48]: from sklearn.metrics import *
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=5)
print(f1_score(y_multilabel, y_train_knn_pred, average="weighted"))
```

```
0.9788245246428662
```

W celu nadania każdej etykiecie wagi równej jej współczynnikowi wsparcia (support) np. liczba próbek zawierających etykietę docelową, można użyć wyrażenia *average="weighted"*.

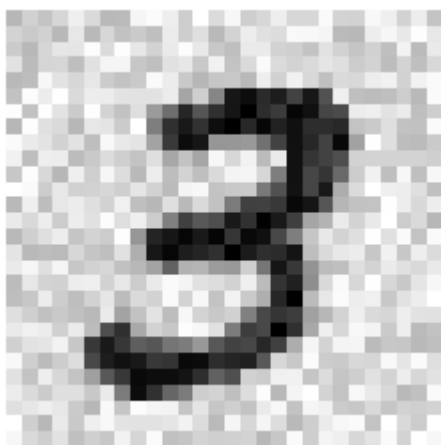
Klasyfikacja wielowyjściowa Ostatnim rodzajem zadania klasyfikacyjnego jest klasyfikacja wielowyjściowa (*multioutput classification*). Jest to po prostu uogólnienie klasyfikacji wieloetykietowej, gdzie każda etykieta może być wieloklasowa tj. może mieć ponad dwie możliwe wartości. Można to przedstawić na systemie, który będzie usuwał szum z obrazów cyfr. Danymi wejściowymi będą poprzednie obrazy, które będą zmodyfikowane poprzez dodanie dodatkowego szumu zaś na wyjściu należy spodziewać się obrazu cyfr w postaci zawierającej poziomy szarości poszczególnych pikseli (na jeden piksel przypada jedna etykieta, a każda z etykiet może mieć różne wartości z zakresu poziomów szarości od 0 do 255)

```
In [49]: noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise
noise = np.random.randint(0, 100, (len(X_test), 784))
X_test_mod = X_test + noise
y_train_mod = X_train
y_test_mod = X_test
```

oraz wyświetlimy tak zmodyfikowany obraz.

```
In [50]: print('Zaszumiony obraz')
plt.imshow(X_train_mod[7].reshape(28,28), cmap = "binary")
plt.axis("off")
plt.show()
print('Prawdziwa etykieta: ', y_train[7])
```

Zaszumiony obraz



Prawdziwa etykieta: 3

Następnie wytrenowujemy klasyfikator na tak zmodyfikowanych danych wejściowych i analizujemy wyniki predykcji

```
In [15]: knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[7]])
plt.imshow(clean_digit.reshape(28,28), cmap = "binary")
plt.axis("off")
plt.show()
print('Prawdziwa etykieta: ', y_train[7])
```



Prawdziwa etykieta: 3

które nie zawsze muszą odzwierciedlać pierwotny stan naszej cyfry wobec wprowadzonego szumu.

Zadania do wykonania (Do wyboru)

Zadanie 1

Wykonaj i oceń wynik klasyfikacji wieloklasowej cyfr ze zbioru MNIST wyłącznie dla cyfr znajdujących się w Twoim numerze albumu.

Zadanie 2

Dla przykładu klasyfikacji wieloetykietowej przeprowadź eksperymenty, aby klasyfikować liczby pierwsze i podzielne przez 3.

Zadanie 3

Dla przykładu klasyfikacji wielowyjściowej wykonaj dwa eksperymenty dla różnych przedziałów zaszumienia oraz przeprowadź testy dla cyfry będącej odpowiednikiem ostatniej cyfry z Twojego numeru albumu.

Rozwiązanie zadań 1-3

Numer albumu: 193257 .

Czyli cyfry brane pod uwagę do zadania 1 to: 1, 2, 3, 5, 7, 9

Do zadania 3 testowana cyfra to 7 .

```
In [ ]: from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import SGDClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    mean_absolute_error,
```

```

    mean_squared_error,
    precision_score,
    recall_score,
)
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

np.set_printoptions(linewidth=160)

RANDOM_STATE = 42
album_number = "193257"
album_digits = sorted({int(digit) for digit in album_number})
last_digit = int(album_number[-1])

mnist_path = Path.home() / ".keras" / "datasets" / "mnist.npz"
with np.load(mnist_path) as data:
    X_train_img = data["x_train"]
    y_train = data["y_train"].astype(np.uint8)
    X_test_img = data["x_test"]
    y_test = data["y_test"].astype(np.uint8)

X_train = X_train_img.reshape(len(X_train_img), -1).astype(np.float32)
X_test = X_test_img.reshape(len(X_test_img), -1).astype(np.float32)

print(f"Cyfry użyte w zadaniu 1: {album_digits}")
print(f>Ostatnia cyfra numeru albumu: {last_digit}")
print(f>Zbiór treningowy: {X_train.shape}, zbiór testowy: {X_test.shape}")

```

Numer albumu: 193257

Cyfry użyte w zadaniu 1: [1, 2, 3, 5, 7, 9]

Ostatnia cyfra numeru albumu: 7

Zbiór treningowy: (60000, 784), zbiór testowy: (10000, 784)

Zadanie 1

Klasyfikacja wieloklasowa -> tylko dla cyfr występujących w indeksie, czyli dla klas 1, 2, 3, 5, 7, 9. Używane jest `SGDClassifier` po standaryzacji danych, bo w przykładzie wcześniej też był używany klasyfikator SGD.

```

In [2]: train_mask = np.isin(y_train, album_digits)
        test_mask = np.isin(y_test, album_digits)

X_train_album = X_train[train_mask]
y_train_album = y_train[train_mask]
X_test_album = X_test[test_mask]
y_test_album = y_test[test_mask]

sgd_album_clf = make_pipeline(
    StandardScaler(),
    SGDClassifier(random_state=RANDOM_STATE, max_iter=1000, tol=1e-3),
)
sgd_album_clf.fit(X_train_album, y_train_album)
y_pred_album = sgd_album_clf.predict(X_test_album)

train_counts = {int(digit): int((y_train_album == digit).sum()) for digit in album_digits}
test_counts = {int(digit): int((y_test_album == digit).sum()) for digit in album_digits}
conf_mx_album = confusion_matrix(y_test_album, y_pred_album, labels=album_digits)

```



```

print(f"Liczba próbek treningowych dla wybranych cyfr: {train_counts}")
print(f"Liczba próbek testowych dla wybranych cyfr: {test_counts}")
print(f"Dokładność na zbiorze testowym: {accuracy_score(y_test_album, y_p
print(f"Macierz pomyłek (wiersze=prawdziwe, kolumny=predykcje; etykiety={
print(conf_mx_album)
print()
print("Raport klasyfikacji:")
print(classification_report(y_test_album, y_pred_album, labels=album_digi

```

Liczba próbek treningowych dla wybranych cyfr: {1: 6742, 2: 5958, 3: 6131, 5: 5421, 7: 6265, 9: 5949}

Liczba próbek testowych dla wybranych cyfr: {1: 1135, 2: 1032, 3: 1010, 5: 892, 7: 1028, 9: 1009}

Dokładność na zbiorze testowym: 0.9135

Macierz pomyłek (wiersze=prawdziwe, kolumny=predykcje; etykiety=[1, 2, 3, 5, 7, 9]):

```

[[1093    2    38    2    0    0]
 [   4  924   86    9    6    3]
 [   0   11  957   25    9    8]
 [   6    8   81  778    6   13]
 [   0   13   65    0  926   24]
 [   6    4   79    4   16  900]]

```

Raport klasyfikacji:

	precision	recall	f1-score	support
1	0.9856	0.9630	0.9742	1135
2	0.9605	0.8953	0.9268	1032
3	0.7328	0.9475	0.8264	1010
5	0.9511	0.8722	0.9099	892
7	0.9616	0.9008	0.9302	1028
9	0.9494	0.8920	0.9198	1009
accuracy			0.9135	6106
macro avg	0.9235	0.9118	0.9145	6106
weighted avg	0.9245	0.9135	0.9159	6106

Wnioski - zadanie 1

Po odfiltrowaniu MNIST do cyfr z mojego numeru albumu, czyli 1, 2, 3, 5, 7, 9, klasyfikator SGD po standaryzacji uzyskał dokładność 0.9135 na 6106

próbkach testowych. Najpewniej rozpoznawana była cyfra 1

(precision=0.9856 , recall=0.9630). Największy problem widać przy cyfrze 3 : ma wysoki recall=0.9475 , ale niższe precision=0.7328 , bo model dość często przypisywał do klasy 3 także inne cyfry, np. 86 dwójek, 81 piątek, 79 dziewiątek i 65 siódemek. Ogólnie wynik jest dobry, ale macierz pomyłek pokazuje, że model za często wybiera klasę 3 .

Zadanie 2

Tu są tworzone dwie etykiety naraz: czy cyfra jest pierwsza (2, 3, 5, 7) oraz czy jest podzielna przez 3 (0, 3, 6, 9). Do eksperymentu używam KNN, tak jak w przykładzie klasyfikacji wieloetykietowej.

```

In [3]: prime_digits = [2, 3, 5, 7]
        divisible_by_3_digits = [0, 3, 6, 9]

```

```

y_train_multilabel = np.c_[
    np.isin(y_train, prime_digits),
    np.isin(y_train, divisible_by_3_digits),
]
y_test_multilabel = np.c_[
    np.isin(y_test, prime_digits),
    np.isin(y_test, divisible_by_3_digits),
]

rng_task2 = np.random.default_rng(RANDOM_STATE)
task2_train_indices = []
for digit in range(10):
    digit_indices = np.flatnonzero(y_train == digit)
    task2_train_indices.extend(rng_task2.choice(digit_indices, size=1200,
task2_train_indices = np.array(task2_train_indices)
rng_task2.shuffle(task2_train_indices)

knn_multilabel_clf = KNeighborsClassifier(
    n_neighbors=3,
    weights="distance",
    algorithm="brute",
    n_jobs=-1,
)
knn_multilabel_clf.fit(X_train[task2_train_indices], y_train_multilabel[t
y_pred_multilabel = knn_multilabel_clf.predict(X_test)

label_names = ["liczba pierwsza", "podzielna przez 3"]
label_accuracy = (y_test_multilabel == y_pred_multilabel).mean(axis=0)
precision_values = precision_score(y_test_multilabel, y_pred_multilabel,
recall_values = recall_score(y_test_multilabel, y_pred_multilabel, averag
f1_values = f1_score(y_test_multilabel, y_pred_multilabel, average=None,

print(f"Użyty podzbiór treningowy: {len(task2_train_indices)} próbek (po
print(f"Zbiór testowy: {len(X_test)} próbek")
print(f"Exact match accuracy: {accuracy_score(y_test_multilabel, y_pred_m
for index, label_name in enumerate(label_names):
    print(
        f"{label_name}: accuracy={label_accuracy[index]:.4f}, "
        f"precision={precision_values[index]:.4f}, "
        f"recall={recall_values[index]:.4f}, "
        f"F1={f1_values[index]:.4f}"
    )
print(f"F1 weighted: {f1_score(y_test_multilabel, y_pred_multilabel, aver
print()
print("Macierz pomyłek - liczba pierwsza:")
print(confusion_matrix(y_test_multilabel[:, 0], y_pred_multilabel[:, 0]))
print("Macierz pomyłek - podzielna przez 3:")
print(confusion_matrix(y_test_multilabel[:, 1], y_pred_multilabel[:, 1]))

```

Użyty podzbiór treningowy: 12000 próbek (po 1200 z każdej cyfry)
Zbiór testowy: 10000 próbek
Exact match accuracy: 0.9617
liczba pierwsza: accuracy=0.9777, precision=0.9780, recall=0.9654, F1=0.9717
podzielna przez 3: accuracy=0.9737, precision=0.9622, recall=0.9717, F1=0.9669
F1 weighted: 0.9693

Macierz pomyłek - liczba pierwsza:

```
[[5952  86]
 [ 137 3825]]
```

Macierz pomyłek - podzielna przez 3:

```
[[5892 151]
 [ 112 3845]]
```

Wnioski - zadanie 2

Dla klasyfikacji wieloetykietowej KNN trenowany na 12000 próbkach dał exact match accuracy=0.9617, więc w większości przypadków trafiał obie etykiety jednocześnie. Etykieta liczba pierwsza wyszła minimalnie lepiej (F1=0.9717) niż etykieta podzielna przez 3 (F1=0.9669), ale różnica jest mała. W macierzy pomyłek dla liczb pierwszych było 86 fałszywych alarmów i 137 pominieć, a dla podzielności przez 3 odpowiednio 151 fałszywych alarmów i 112 pominieć. Wyniki są dobre, bo obie etykiety mają accuracy powyżej 0.97.

Zadanie 3

Dla klasyfikacji wielowyjściowej robię dwa eksperymenty odsumowania: szum z przedziału 0-50 oraz 0-150. Testuję cyfrę 7, bo to ostatnia cyfra numeru albumu. W tym zadaniu każdy piksel traktuję jako osobne wyjście klasyfikatora KNN.

```
In [4]: rng_task3 = np.random.default_rng(RANDOM_STATE)
task3_train_indices = []
for digit in range(10):
    digit_indices = np.flatnonzero(y_train == digit)
    task3_train_indices.extend(rng_task3.choice(digit_indices, size=800,
    task3_train_indices = np.array(task3_train_indices)
    rng_task3.shuffle(task3_train_indices)

X_train_clean_small = X_train[task3_train_indices].astype(np.uint8)
digit_test_indices = np.flatnonzero(y_test == last_digit)[:100]
X_test_digit_clean = X_test[digit_test_indices].astype(np.uint8)

noise_ranges = [(0, 50), (0, 150)]
denoising_results = []
example_images = []

for low, high in noise_ranges:
    local_rng = np.random.default_rng(RANDOM_STATE + high)
    train_noise = local_rng.integers(low, high + 1, size=X_train_clean_sm
    test_noise = local_rng.integers(low, high + 1, size=X_test_digit_clea

    X_train_noisy = np.clip(X_train_clean_small.astype(np.uint16) + train
    X_test_digit_noisy = np.clip(X_test_digit_clean.astype(np.uint16) + t
```

```

knn_denoiser = KNeighborsClassifier(
    n_neighbors=3,
    weights="distance",
    algorithm="brute",
    n_jobs=-1,
)
knn_denoiser.fit(X_train_noisy, X_train_clean_small)
X_digit_denoised = knn_denoiser.predict(X_test_digit_noisy).astype(np

clean = X_test_digit_clean.astype(np.float32)
noisy = X_test_digit_noisy.astype(np.float32)
denoising_results.append(
    {
        "noise_range": f"{low}-{high}",
        "noisy_mse": mean_squared_error(clean, noisy),
        "denoised_mse": mean_squared_error(clean, X_digit_denoised),
        "noisy_mae": mean_absolute_error(clean, noisy),
        "denoised_mae": mean_absolute_error(clean, X_digit_denoised),
        "first_digit_noisy_mse": mean_squared_error(clean[0], noisy[0]
        "first_digit_denoised_mse": mean_squared_error(clean[0], X_di
    }
)
example_images.append((f"{low}-{high}", clean[0], noisy[0], X_digit_d

print(f"Użyty podzbiór treningowy: {len(task3_train_indices)} próbek (po
print(f"Testowana cyfra: {last_digit}, liczba testowanych obrazów: {len(d
for result in denoising_results:
    mse_change = (1 - result["denoised_mse"] / result["noisy_mse"]) * 100
    print(
        f"Szum {result['noise_range']}: "
        f"MSE przed={result['noisy_mse']:.2f}, MSE po={result['denoised_m
        f"MAE przed={result['noisy_mae']:.2f}, MAE po={result['denoised_m
        f"zmiana MSE={mse_change:.2f}%"
    )
    print(
        f"Pierwsza testowana 7: MSE przed={result['first_digit_noisy_mse']
        f"MSE po={result['first_digit_denoised_mse']:.2f}"
    )

fig, axes = plt.subplots(len(example_images), 3, figsize=(8, 5))
for row, (noise_label, clean_image, noisy_image, denoised_image) in enumerate
    axes[row, 0].imshow(clean_image.reshape(28, 28), cmap="binary")
    axes[row, 0].set_title(f"Oryginał 7")
    axes[row, 1].imshow(noisy_image.reshape(28, 28), cmap="binary")
    axes[row, 1].set_title(f"Szum {noise_label}")
    axes[row, 2].imshow(denoised_image.reshape(28, 28), cmap="binary")
    axes[row, 2].set_title("Po KNN")
    for col in range(3):
        axes[row, col].axis("off")
plt.tight_layout()
plt.show()

```

Użyty podzbiór treningowy: 8000 próbek (po 800 z każdej cyfry)

Testowana cyfra: 7, liczba testowanych obrazów: 100

Szum 0-50: MSE przed=778.68, MSE po=1594.20, MAE przed=23.39, MAE po=11.19, zmiana MSE=-104.73%

Pierwsza testowana 7: MSE przed=804.28, MSE po=510.99

Szum 0-150: MSE przed=6827.54, MSE po=1775.59, MAE przed=69.00, MAE po=12.03, zmiana MSE=73.99%

Pierwsza testowana 7: MSE przed=6848.63, MSE po=510.99

Wnioski - zadanie 3

Dla cyfry 7 mocniejszy szum 0-150 został odszumiony wyraźnie lepiej: średni MSE spadł z 6827.54 do 1775.59, a MAE z 69.00 do 12.03. Przy słabszym szumie 0-50 wynik jest mieszany: MAE poprawił się z 23.39 do 11.19, ale MSE pogorszył się z 778.68 do 1594.20. Dla pierwszej testowanej siódemki odszumianie poprawiło MSE w obu eksperymentach (804.28 -> 510.99 oraz 6848.63 -> 510.99). Wyszło więc, że KNN pomaga szczególnie przy mocnym zaszumieniu, a przy lekkim szumie potrafi zmienić kształt cyfry na tyle, że średni błąd kwadratowy może wyjść gorszy.

stopka.svg